Battery Degradation-Aware PV-BESS Energy Management and Sizing for Munich Households

Oihika Arpit

Case Study: Munich, Germany

Latitude: 48.1351 N, Longitude: 11.5820 E

Abstract

Abstract--This project presents a degradation-aware photovoltaic battery energy storage system simulation for residential households in Munich, Germany. The model evaluates hourly PV generation, household demand, battery state of charge, depth of discharge, degradation over 15 years, battery replacement timing, payback period, levelized cost of energy, battery sizing, second-life EV battery performance, and frequency support.

Keywords

Index Terms--PV-BESS, battery degradation, energy management, Munich, second-life EV battery, LCOE, payback, frequency support.

I. Introduction and II. Methodology

I. INTRODUCTION

Residential PV-BESS systems can reduce grid imports and improve self-consumption, but battery aging changes both technical and economic performance over time. A degradation-aware model is therefore required for realistic sizing and cost assessment.

This study focuses on Munich, Germany, using a synthetic hourly household load profile and a Munich-specific PV yield assumption.

II. METHODOLOGY

The model performs hourly PV-BESS dispatch for 15 years. PV generation first supplies household load. Surplus PV charges the battery, and remaining energy is exported. During deficits, the battery discharges before grid import.

Battery aging is represented using calendar degradation and cycle degradation. Replacement is triggered when state of health falls below 70%. Economic indicators include annual savings, discounted cash flow, payback year, and LCOE.

III. FEATURES INCLUDED

- PV-BESS energy management simulation
- SOC tracking and depth of discharge analysis
- 15-year degradation and replacement timing
- Payback and LCOE calculation
- Battery sizing optimizer
- New Li-ion vs second-life EV battery comparison
- Simple frequency support simulation

IV. System Parameters

Item	Value
Location	Munich, Germany
Author	Oihika Arpit
Annual household load	4500 kWh/year
PV system size	6 kWp
Munich PV yield	1050 kWh/kWp/year
Grid electricity price	0.40 EUR/kWh
Feed-in tariff	0.08 EUR/kWh
Discount rate	4%
PV capital cost	1300 EUR/kWp
PV O&M	120 EUR/year
Battery replacement threshold	70% SOH
Simulation period	15 years

V. Battery Cases and Optimized Result

Item	Value
New Li-ion	650 EUR/kWh, 100% initial SOH, 92% efficiency, 90% max
Second-life EV	330 EUR/kWh, 82% initial SOH, 88% efficiency, 80% max D
Best battery type	Second-life EV
Best battery size	4 kWh
Best LCOE	0.400 EUR/kWh
Payback year	11
Replacement year	4
Initial CAPEX	9120 EUR
Year-15 annual savings	1150.38 EUR

VI. Top Output Data by LCOE

Item	Value
Second-life EV, 4 kWh	LCOE 0.400 EUR/kWh, payback 11, replacement 4
Second-life EV, 6 kWh	LCOE 0.400 EUR/kWh, payback 11, replacement 4
Second-life EV, 2 kWh	LCOE 0.401 EUR/kWh, payback 10, replacement 4
Second-life EV, 8 kWh	LCOE 0.404 EUR/kWh, payback 12, replacement 4
New Li-ion, 2 kWh	LCOE 0.406 EUR/kWh, payback 11, replacement 10
New Li-ion, 4 kWh	LCOE 0.410 EUR/kWh, payback 12, replacement 11

Fig. 1. Munich PV and Household Load Profile

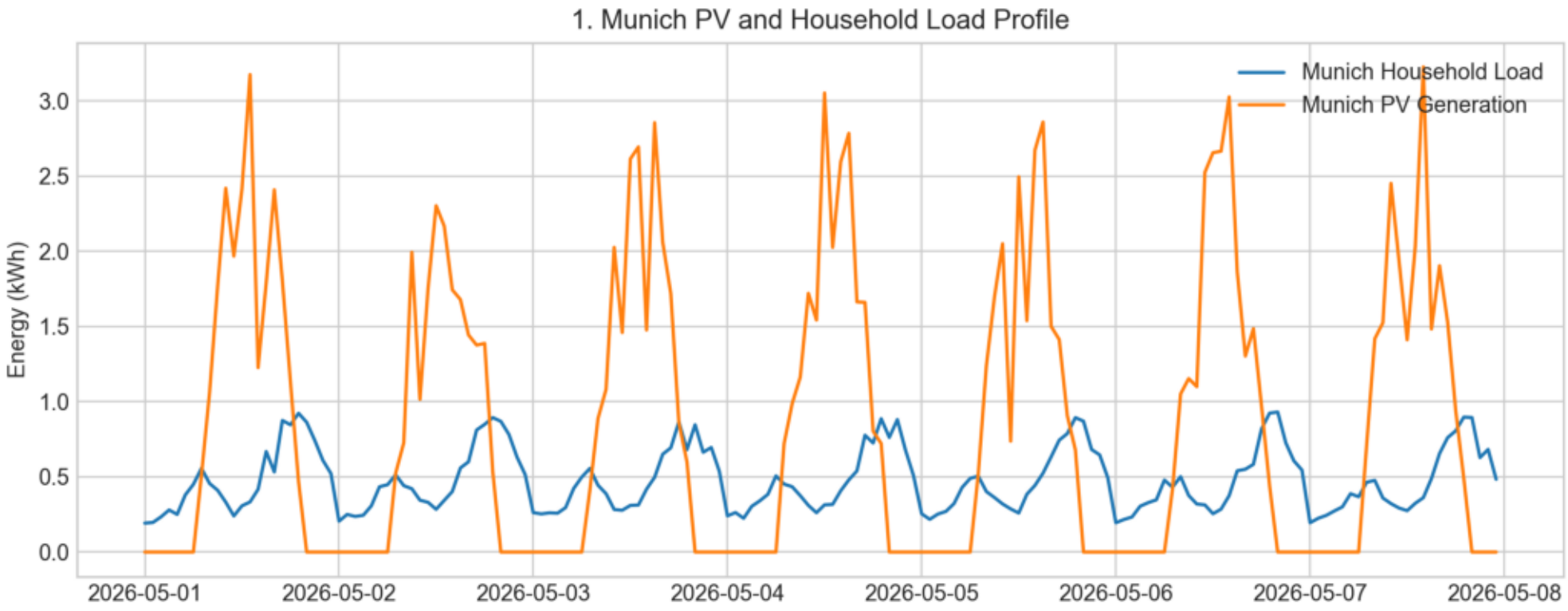


Fig. 2. Battery SOC Tracking in Munich

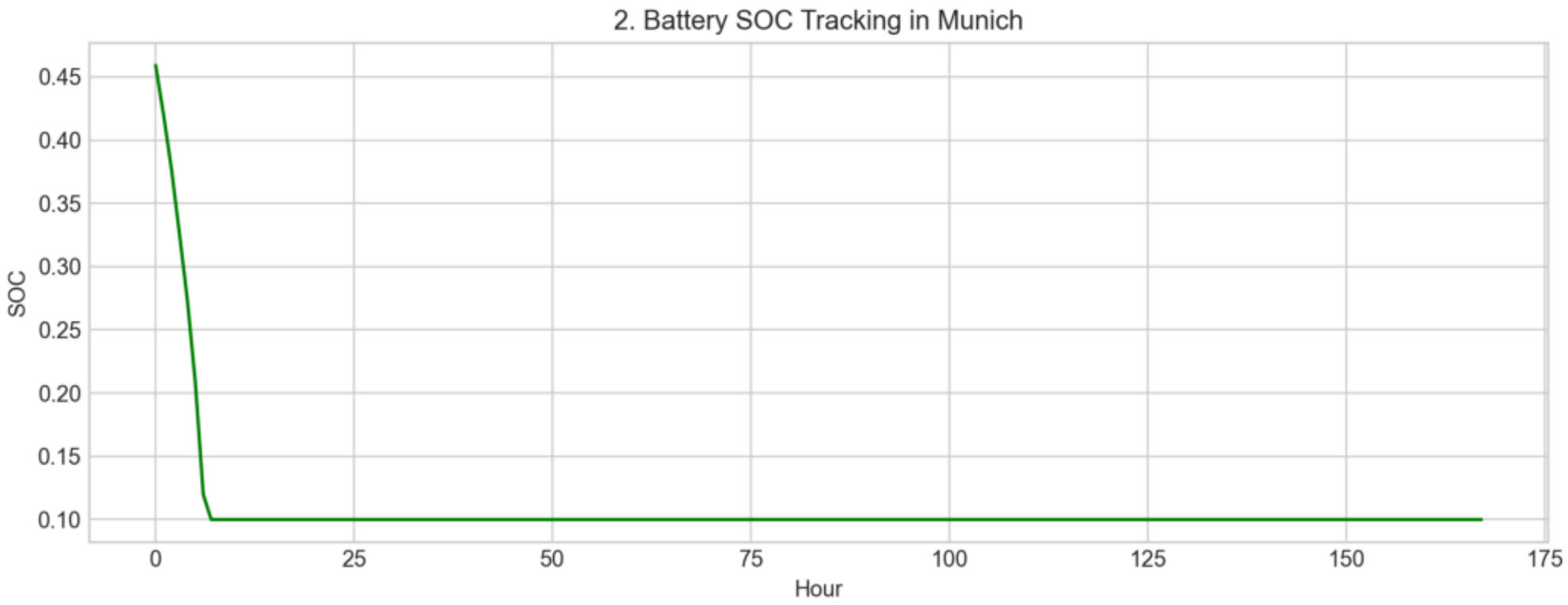


Fig. 3. Battery Depth of Discharge in Munich

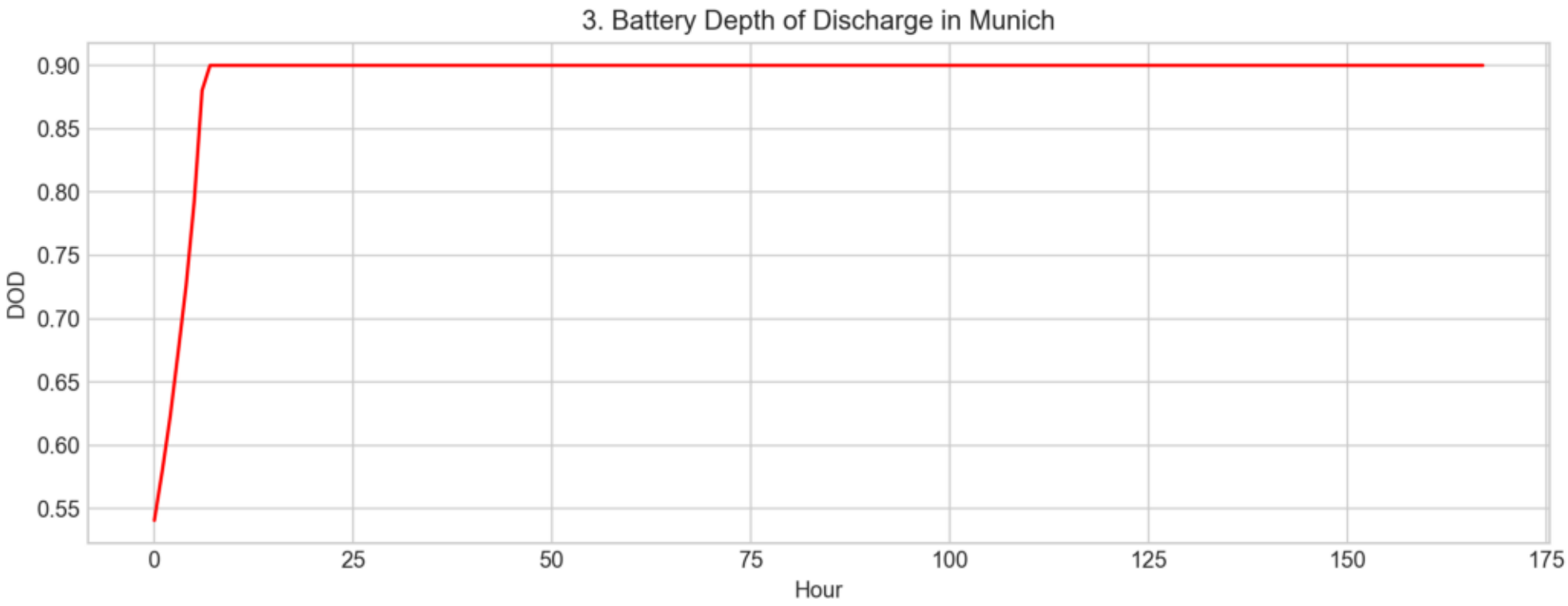


Fig. 4. Battery Degradation Over 15 Years

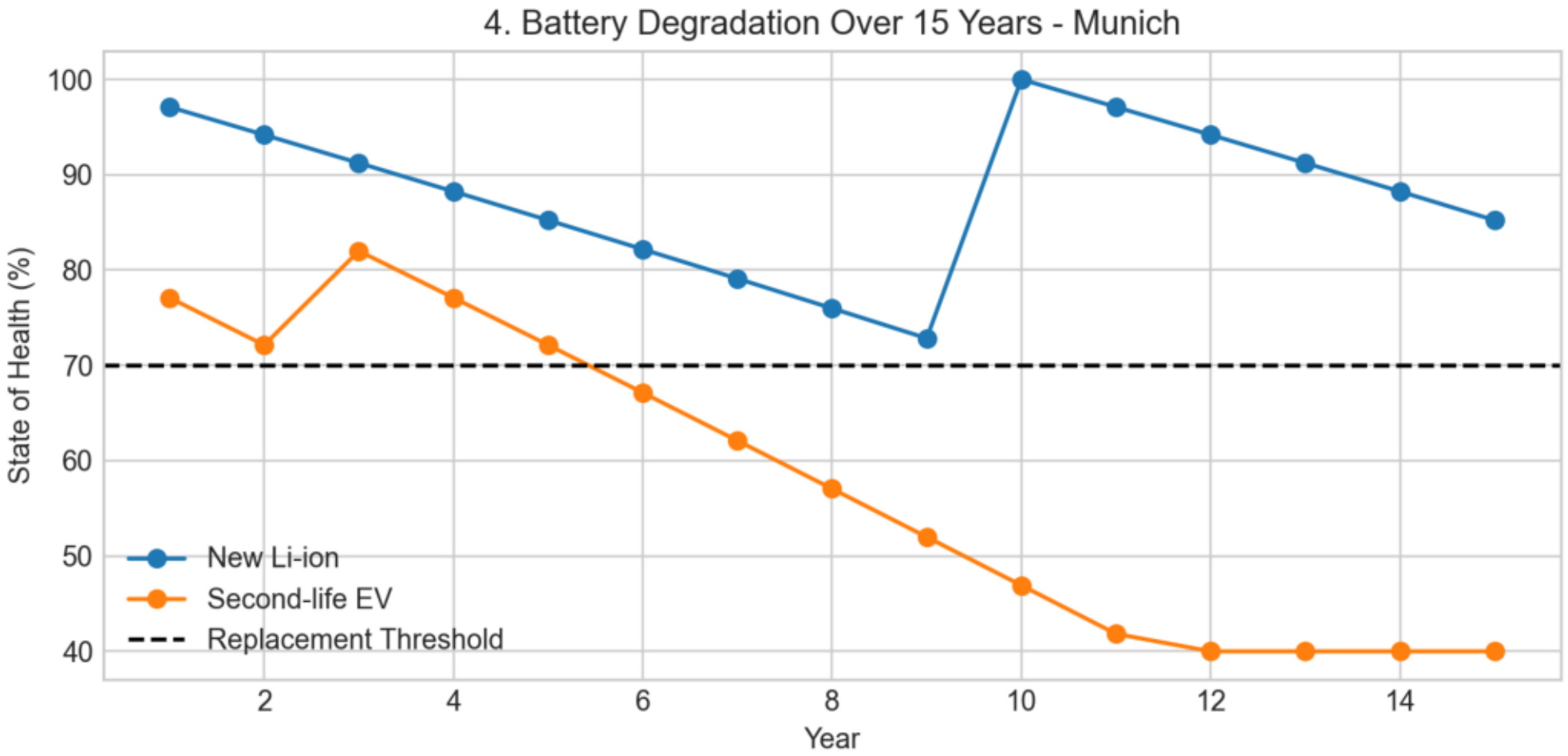


Fig. 5. Battery Replacement Year

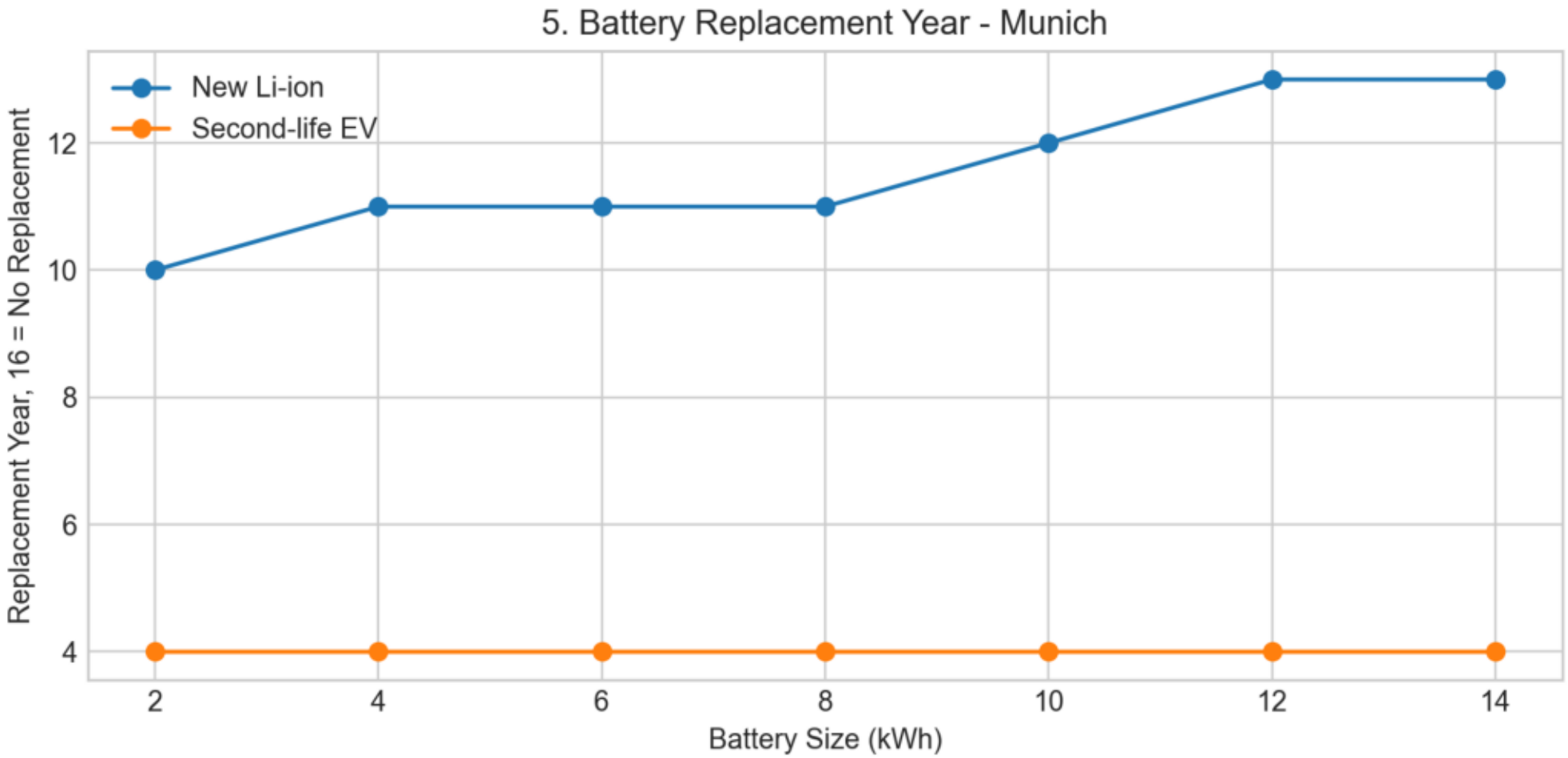


Fig. 6. Payback Year

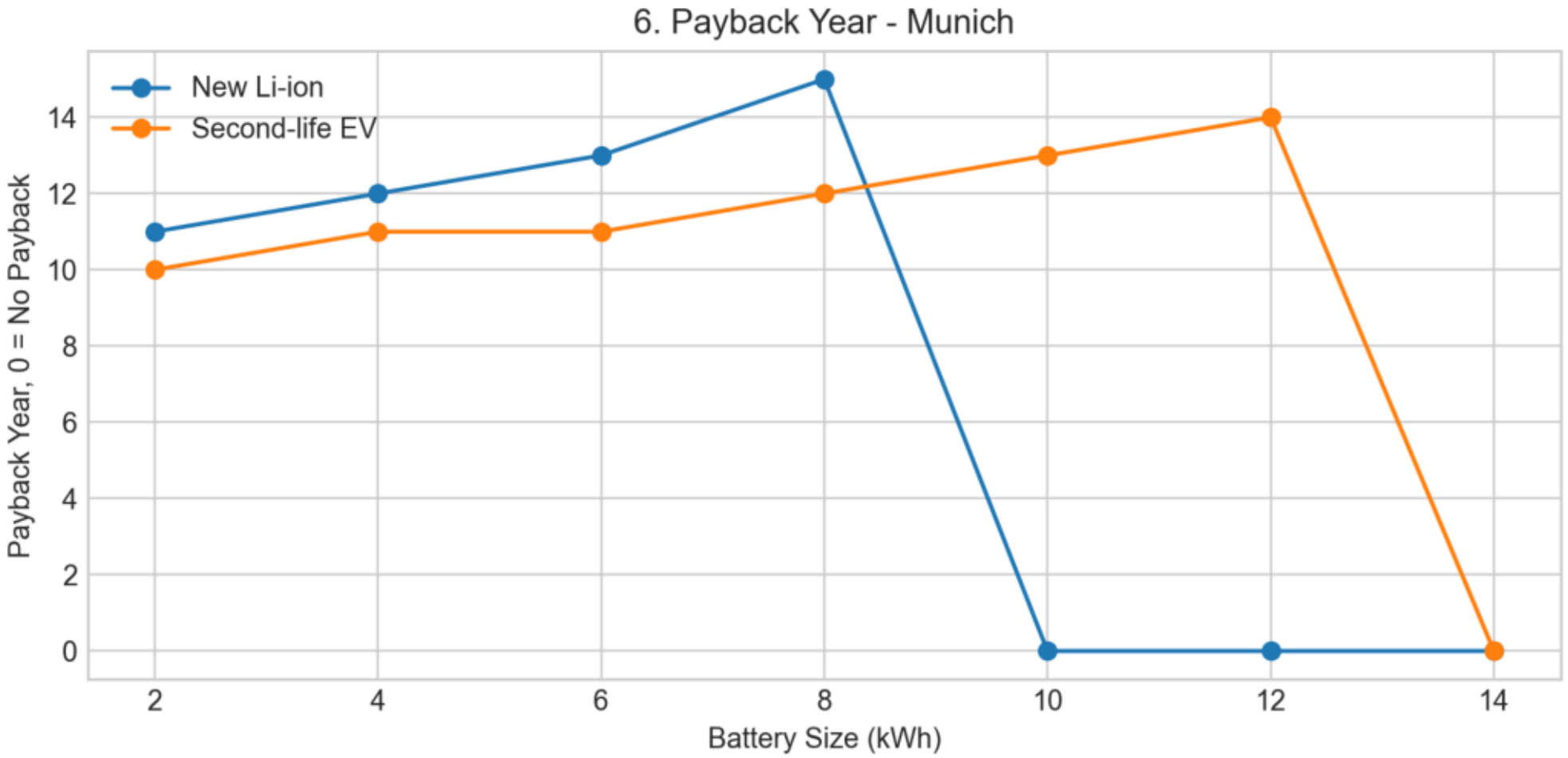


Fig. 7. LCOE Impact

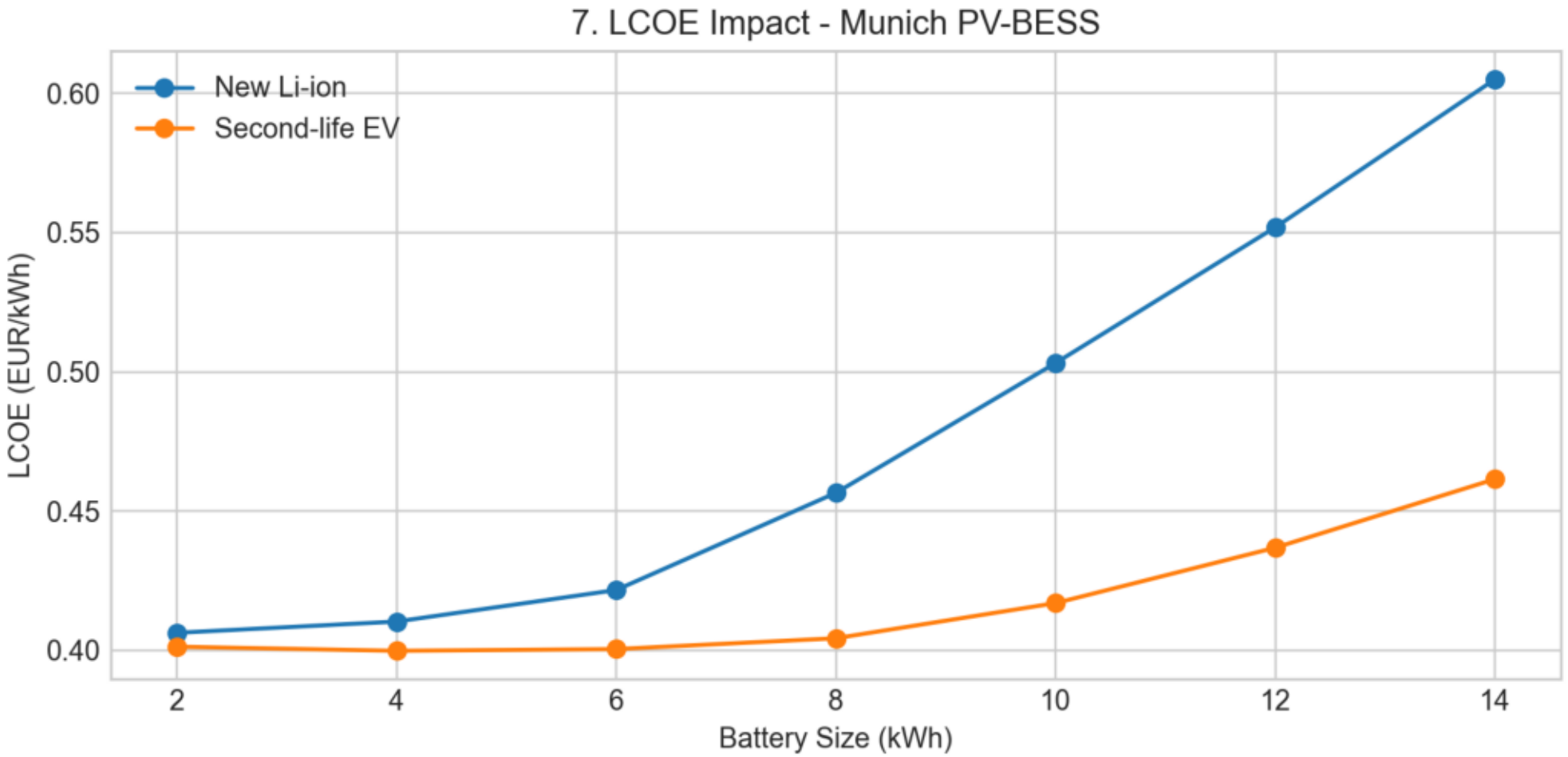


Fig. 8. New vs Second-Life EV Battery CAPEX

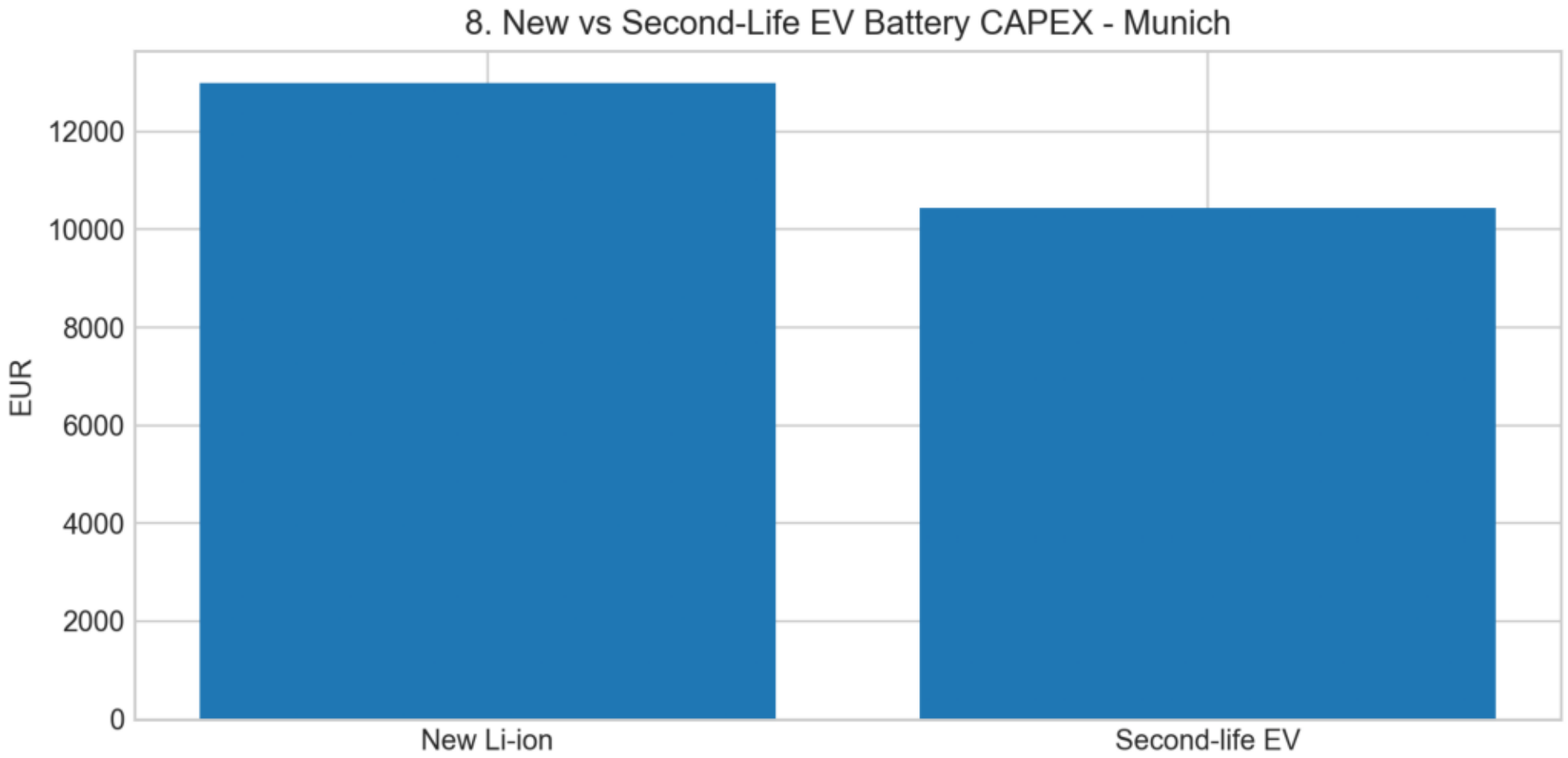
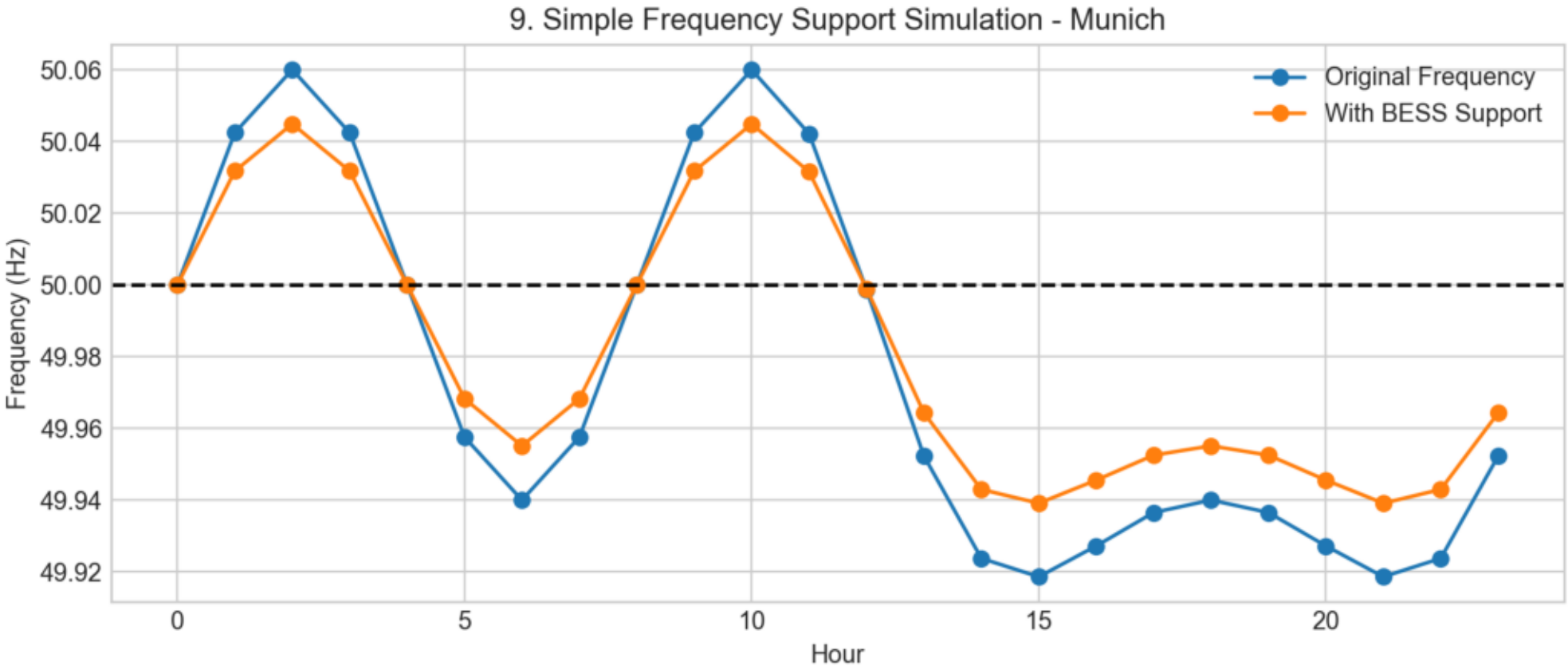


Fig. 9. Simple Frequency Support Simulation



VII. Discussion and VIII. Conclusion

VII. DISCUSSION

The results show that battery degradation significantly affects long-term PV-BESS economics. Larger batteries increase self-consumption but also increase capital cost. Smaller batteries have lower cost but limited storage capability.

The second-life EV battery option achieves the lowest LCOE in this case because its lower purchase cost compensates for lower initial SOH and faster degradation. However, replacement planning is important because the second-life battery reaches the replacement threshold earlier.

VIII. CONCLUSION

A degradation-aware PV-BESS simulation was developed for Munich households. The model includes SOC, DOD, battery aging, replacement timing, payback, LCOE, battery sizing, second-life EV battery comparison, and frequency support.

For the selected assumptions, the optimized result is a 4 kWh second-life EV battery with an LCOE of approximately 0.400 EUR/kWh and payback in year 11.

Appendix A: Python Simulation Code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from pathlib import Path

YEARS = 15
HOURS = 8760

CITY = "Munich, Germany"
LATITUDE = 48.1351
LONGITUDE = 11.5820

annual_load_kwh = 4500
pv_size_kwp = 6
munich_pv_yield = 1050
grid_price = 0.40
feed_in_tariff = 0.08
discount_rate = 0.04

pv_capex_per_kwp = 1300
pv_om_per_year = 120

battery_sizes = np.arange(2, 15, 2)
base_battery_size = 8
replacement_threshold = 0.70

battery_types = {
    "New Li-ion": {
        "cost": 650,
        "initial_soh": 1.00,
        "calendar_fade": 0.018,
        "cycle_fade": 0.000055,
        "efficiency": 0.92,
        "max_dod": 0.90,
    },
    "Second-life EV": {
        "cost": 330,
        "initial_soh": 0.82,
        "calendar_fade": 0.030,
        "cycle_fade": 0.000085,
        "efficiency": 0.88,
        "max_dod": 0.80,
    },
}

pv_capex = pv_size_kwp * pv_capex_per_kwp


def create_munich_profiles():
    np.random.seed(42)
    time = pd.date_range("2026-01-01", periods=HOURS, freq="h")
    hour = time.hour.values
    day = time.dayofyear.values

```

Appendix A: Python Simulation Code

```
    morning_peak = 0.4 * np.exp(-0.5 * ((hour - 7) / 2) ** 2)
    evening_peak = 1.0 * np.exp(-0.5 * ((hour - 19) / 3) ** 2)
    winter_factor = 1 + 0.22 * np.cos(2 * np.pi * (day - 15) / 365)

    load = 0.35 + morning_peak + evening_peak
    load = load * winter_factor * np.random.normal(1, 0.08, HOURS)
    load = load * annual_load_kwh / load.sum()

    daylight = np.maximum(0, np.sin(np.pi * (hour - 6) / 14))
    seasonal_factor = np.maximum(0.08, np.sin(np.pi * (day - 20) / 365))
    cloud_factor = np.clip(np.random.normal(0.92, 0.20, HOURS), 0.25, 1.25)

    pv = daylight * seasonal_factor * cloud_factor
    pv = pv * (pv_size_kwp * munich_pv_yield) / pv.sum()

    return pd.DataFrame({"load": load, "pv": pv}, index=time)


def simulate_bess(profile, battery_size, battery_name, keep_hourly=False):
    battery = battery_types[battery_name]
    soh = battery["initial_soh"]
    efficiency = battery["efficiency"]
    charge_eff = np.sqrt(efficiency)
    discharge_eff = np.sqrt(efficiency)
    min_soc_fraction = 1 - battery["max_dod"]
    soc = 0.5 * battery_size * soh

    load_values = profile["load"].to_numpy()
    pv_values = profile["pv"].to_numpy()
    timestamps = profile.index.to_numpy()

    yearly_results = []
    hourly_results = []
    initial_capex = pv_capex + battery_size * battery["cost"]
    cumulative_cashflow = -initial_capex
    replacement_year = None

    for year in range(1, YEARS + 1):
        usable_capacity = battery_size * soh
        min_soc = min_soc_fraction * usable_capacity
        max_soc = usable_capacity
        soc = np.clip(soc, min_soc, max_soc)

        grid_import = 0
        grid_export = 0
        self_consumed = 0
        battery_throughput = 0
        soc_year = []
        dod_year = []

        for i in range(len(load_values)):
            load = load_values[i]
            pv = pv_values[i]
            surplus = pv - load
```

Appendix A: Python Simulation Code

```
            if surplus >= 0:
                charge = min(surplus * charge_eff, max_soc - soc)
                soc += charge
                export = surplus - charge / charge_eff
                import_energy = 0
                self_consumed += load
                battery_throughput += charge
            else:
                deficit = -surplus
                discharge = min(deficit / discharge_eff, soc - min_soc)
                soc -= discharge
                import_energy = deficit - discharge * discharge_eff
                export = 0
                self_consumed += pv + discharge * discharge_eff
                battery_throughput += discharge

            grid_import += import_energy
            grid_export += export

            soc_fraction = soc / max_soc
            dod_fraction = 1 - soc_fraction
            soc_year.append(soc_fraction)
            dod_year.append(dod_fraction)

            if keep_hourly and year == 1:
                hourly_results.append({
                    "timestamp": timestamps[i],
                    "load": load,
                    "pv": pv,
                    "soc": soc_fraction,
                    "dod": dod_fraction,
                    "grid_import": import_energy,
                    "grid_export": export,
                })

        equivalent_cycles = battery_throughput / (2 * usable_capacity)
        degradation = battery["calendar_fade"] + battery["cycle_fade"] * equivalent_cycles
        soh_next = max(0.40, soh - degradation)

        replacement_cost = 0
        if soh_next < replacement_threshold and replacement_year is None and year < YEARS:
            replacement_year = year + 1
            replacement_cost = battery_size * battery["cost"]
            soh_next = battery["initial_soh"]

        baseline_cost = annual_load_kwh * grid_price
        annual_bill = (
            grid_import * grid_price
            - grid_export * feed_in_tariff
            + pv_om_per_year
            + replacement_cost
        )
        annual_savings = baseline_cost - annual_bill
```

Appendix A: Python Simulation Code

```
        cumulative_cashflow += annual_savings / ((1 + discount_rate) ** year)

        yearly_results.append({
            "city": CITY,
            "latitude": LATITUDE,
            "longitude": LONGITUDE,
            "battery": battery_name,
            "battery_size": battery_size,
            "year": year,
            "soh": soh_next,
            "mean_soc": np.mean(soc_year),
            "mean_dod": np.mean(dod_year),
            "max_dod": np.max(dod_year),
            "cycles": equivalent_cycles,
            "grid_import": grid_import,
            "grid_export": grid_export,
            "self_consumed": self_consumed,
            "annual_savings": annual_savings,
            "cashflow": cumulative_cashflow,
            "replacement_year": replacement_year if replacement_year else 16,
            "replacement_cost": replacement_cost,
            "capex": initial_capex,
        })

        soh = soh_next

    return pd.DataFrame(yearly_results), pd.DataFrame(hourly_results)


def calculate_lcoe_payback(results):
    updated_results = []

    for (battery_name, size), group in results.groupby(["battery", "battery_size"]):
        battery = battery_types[battery_name]
        capex = pv_capex + size * battery["cost"]
        discounted_cost = capex
        discounted_energy = 0
        payback = 0

        for _, row in group.iterrows():
            year = row["year"]
            discounted_cost += (
                pv_om_per_year + row["replacement_cost"]
            ) / ((1 + discount_rate) ** year)
            discounted_energy += row["self_consumed"] / ((1 + discount_rate) ** year)

            if payback == 0 and row["cashflow"] >= 0:
                payback = year

        lcoe = discounted_cost / discounted_energy
        temp = group.copy()
        temp["lcoe"] = lcoe
        temp["payback"] = payback
        updated_results.append(temp)
```

Appendix A: Python Simulation Code

```
0217
0218      return pd.concat(updated_results, ignore_index=True)
0219
0220
0221  def frequency_support_simulation():
0222      hours = np.arange(24)
0223      frequency = (
0224          50
0225          + 0.06 * np.sin(2 * np.pi * hours / 8)
0226          - 0.12 * np.exp(-0.5 * ((hours - 18) / 2) ** 2)
0227      )
0228      support_power = np.clip((50 - frequency) * base_battery_size * 0.9, -2.5, 2.5)
0229      corrected_frequency = frequency + 0.035 * support_power
0230
0231      return pd.DataFrame({
0232          "hour": hours,
0233          "frequency": frequency,
0234          "support_power": support_power,
0235          "corrected_frequency": corrected_frequency,
0236      })
0237
0238
0239  def save_plot(path):
0240      plt.tight_layout()
0241      plt.savefig(path, dpi=220)
0242      plt.close()
0243
0244
0245  def main():
0246      output_dir = Path(__file__).resolve().parent
0247      plot_dir = output_dir / "munich_output_images"
0248      plot_dir.mkdir(exist_ok=True)
0249
0250      profile = create_munich_profiles()
0251      all_results = []
0252      base_hourly = None
0253
0254      for battery in battery_types:
0255          for size in battery_sizes:
0256              yearly, hourly = simulate_bess(
0257                  profile,
0258                  size,
0259                  battery,
0260                  keep_hourly=(battery == "New Li-ion" and size == base_battery_size),
0261              )
0262              all_results.append(yearly)
0263              if not hourly.empty:
0264                  base_hourly = hourly
0265
0266      results = calculate_lcoe_payback(pd.concat(all_results, ignore_index=True))
0267      final_results = results[results["year"] == YEARS]
0268      frequency_data = frequency_support_simulation()
0269
0270      profile.to_csv(output_dir / "munich_hourly_profile.csv")
```

Appendix A: Python Simulation Code

```
    results.to_csv(output_dir / "munich_results_summary.csv", index=False)
    final_results.to_csv(output_dir / "munich_final_results.csv", index=False)
    frequency_data.to_csv(output_dir / "munich_frequency_support.csv", index=False)
    base_hourly.to_csv(output_dir / "munich_base_hourly_soc_dod.csv", index=False)

    plt.style.use("seaborn-v0_8-whitegrid")

    week = profile.iloc[24 * 120:24 * 127]
    plt.figure(figsize=(10, 4))
    plt.plot(week.index, week["load"], label="Munich Household Load")
    plt.plot(week.index, week["pv"], label="Munich PV Generation")
    plt.title("1. Munich PV and Household Load Profile")
    plt.ylabel("Energy (kWh)")
    plt.legend()
    save_plot(plot_dir / "01_munich_pv_load_profile.png")

    plt.figure(figsize=(10, 4))
    plt.plot(base_hourly["soc"].iloc[:24 * 7], color="green")
    plt.title("2. Battery SOC Tracking in Munich")
    plt.xlabel("Hour")
    plt.ylabel("SOC")
    save_plot(plot_dir / "02_munich_soc_tracking.png")

    plt.figure(figsize=(10, 4))
    plt.plot(base_hourly["dod"].iloc[:24 * 7], color="red")
    plt.title("3. Battery Depth of Discharge in Munich")
    plt.xlabel("Hour")
    plt.ylabel("DOD")
    save_plot(plot_dir / "03_munich_depth_of_discharge.png")

    plt.figure(figsize=(8, 4))
    for battery in battery_types:
        data = results[
            (results["battery"] == battery)
            & (results["battery_size"] == base_battery_size)
        ]
        plt.plot(data["year"], data["soh"] * 100, marker="o", label=battery)
    plt.axhline(
        replacement_threshold * 100,
        linestyle="--",
        color="black",
        label="Replacement Threshold",
    )
    plt.title("4. Battery Degradation Over 15 Years - Munich")
    plt.xlabel("Year")
    plt.ylabel("State of Health (%)")
    plt.legend()
    save_plot(plot_dir / "04_munich_battery_degradation.png")

    plt.figure(figsize=(8, 4))
    for battery in battery_types:
        data = final_results[final_results["battery"] == battery]
        plt.plot(data["battery_size"], data["replacement_year"], marker="o", label=battery)
    plt.title("5. Battery Replacement Year - Munich")
```

Appendix A: Python Simulation Code

```
    plt.xlabel("Battery Size (kWh)")
    plt.ylabel("Replacement Year, 16 = No Replacement")
    plt.legend()
    save_plot(plot_dir / "05_munich_replacement_year.png")

    plt.figure(figsize=(8, 4))
    for battery in battery_types:
        data = final_results[final_results["battery"] == battery]
        plt.plot(data["battery_size"], data["payback"], marker="o", label=battery)
    plt.title("6. Payback Year - Munich")
    plt.xlabel("Battery Size (kWh)")
    plt.ylabel("Payback Year, 0 = No Payback")
    plt.legend()
    save_plot(plot_dir / "06_munich_payback_year.png")

    plt.figure(figsize=(8, 4))
    for battery in battery_types:
        data = final_results[final_results["battery"] == battery]
        plt.plot(data["battery_size"], data["lcoe"], marker="o", label=battery)
    plt.title("7. LCOE Impact - Munich PV-BESS")
    plt.xlabel("Battery Size (kWh)")
    plt.ylabel("LCOE (EUR/kWh)")
    plt.legend()
    save_plot(plot_dir / "07_munich_lcoe_impact.png")

    comparison = final_results[final_results["battery_size"] == base_battery_size]
    x = np.arange(len(comparison))
    plt.figure(figsize=(8, 4))
    plt.bar(x, comparison["capex"])
    plt.xticks(x, comparison["battery"])
    plt.title("8. New vs Second-Life EV Battery CAPEX - Munich")
    plt.ylabel("EUR")
    save_plot(plot_dir / "08_munich_new_vs_second_life_capex.png")

    plt.figure(figsize=(9, 4))
    plt.plot(
        frequency_data["hour"],
        frequency_data["frequency"],
        marker="o",
        label="Original Frequency",
    )
    plt.plot(
        frequency_data["hour"],
        frequency_data["corrected_frequency"],
        marker="o",
        label="With BESS Support",
    )
    plt.axhline(50, linestyle="--", color="black")
    plt.title("9. Simple Frequency Support Simulation - Munich")
    plt.xlabel("Hour")
    plt.ylabel("Frequency (Hz)")
    plt.legend()
    save_plot(plot_dir / "09_munich_frequency_support.png")

```

Appendix A: Python Simulation Code

```
    best = final_results.sort_values("lcoe").iloc[0]
    print("Munich outputs created.")
    print("Best option:", best["battery"], best["battery_size"], "kWh")
    print("LCOE:", round(best["lcoe"], 3), "EUR/kWh")
    print("Payback:", int(best["payback"]))


if __name__ == "__main__":
    main()
```